Multi-Machine Network Configuration for ROS 2 Jazzy on Ubuntu 24.04

This tutorial documents how the Sigyn robot network is configured across its three primary computers and explains *why* each decision was made. It is written for ROS 2 developers who are comfortable with Linux but may not have deep networking experience.

---

1. The Network Topology

```
Internet
   │
192.168.12.1  ← Google Nest mesh router (primary) + 4 x mesh repea
   │
   ├── wired Ethernet
   │       └── amdc  (192.168.12.140)   AMD Ryzen 9 7900X developme
   │
   └── Wi-Fi 802.11ax (Wi-Fi 6), 5 GHz band
           ├── sigyn7900a  (192.168.12.210)   Robot main computer —
           └── sigynVision (192.168.12.120)   Robot vision processo
```

All three machines share the `192.168.12.0/24` subnet, served by the Google Nest mesh at gateway `192.168.12.1`. There is no NAT between them — they communicate directly at LAN speeds.

---

2. Why Static IPs Instead of DHCP

ROS 2 does not use zeroconf (mDNS) for inter-machine discovery in this setup. All hostnames are resolved through `/etc/hosts` on each machine, and SSH configurations reference machines by their hostnames. This means every machine **must** have a predictable, fixed IP address.

If a machine receives a different DHCP lease after a reboot, every other robot computer loses the ability to reach it until `/etc/hosts` is manually updated everywhere — which is impractical in a fleet.

Rule: every robot computer has a static IP assigned directly in its NetworkManager connection profile (`ipv4.method = manual`). No DHCP

reservations are used because that would create a hidden dependency on the router's config surviving power cycles, firmware updates, or router replacement.

---

3. The `/etc/hosts` File (All Machines)

Each machine carries an identical `/etc/hosts` file covering all robot computers:

```
127.0.0.1       localhost
127.0.1.1       <this-machine-hostname>

192.168.12.136  MiniMe4     MiniMe4    # development/test system
192.168.12.140  amdc        amdc       # AMD Ryzen 9 7900X desktop
192.168.12.210  sigyn7900a  SR         # Robot main computer
192.168.12.120  sigynVision SV         # Robot vision processor (R
192.168.12.219  sigynNvidia1 SN1       # Jetson Orin Nano dev boar
```

The short aliases (`SR`, `SV`, etc.) allow terse usage in scripts. When you add a new robot computer, add its entry to **all** machines' `/etc/hosts`.

---

4. Machine-by-Machine Configuration

4.1 `amdc` — Development Desktop (Wired)

Property	Value
Interface	`wlp9s0` (now **wired** via Ethernet)
IP	`192.168.12.140/24`
Gateway	`192.168.12.1`
Method	Static (`ipv4.method = manual`)
Role	iperf3 server, ROS 2 visualization, bag recording

`amdc` is a desktop: it never moves, so Wi-Fi adds no benefit. Connecting it via Ethernet to the nearest Google Nest point eliminates one wireless hop from the robot→desktop video path. See §7 for why this matters.

4.2 `sigyn7900a` — Robot Main Computer (Wi-Fi)

Property	Value
Interface	`wlp8s0`

Wi-Fi chip	MediaTek MT7922 (PCIe, `mt7921e` driver)
IP	`192.168.12.210/24`
Gateway	`192.168.12.1`
Band	5 GHz only (`802-11-wireless.band = a`)
Channel	Any (`802-11-wireless.channel = 0`)
Powersave	Disabled (`802-11-wireless.powersave = 2`)
MAC randomization	Off (`mac-address-randomization = never`)
Route metric	50 (lowest = preferred default route)

4.3 `sigynVision` — Robot Vision Processor (Wi-Fi)

Property	Value
Interface	`wlan0`
Wi-Fi chip	Broadcom CYW43 on-board (SDIO, `brcmfmac` driver)
IP	`192.168.12.120/24`
Gateway	`192.168.12.1`
Band	5 GHz only (`802-11-wireless.band = a`)
Channel	Any (`802-11-wireless.channel = 0`)
Powersave	Disabled
MAC randomization	Off
Route metric	50

`sigynVision` is a Raspberry Pi 5 mounted on the robot's gripper assembly. It is always co-located with `sigyn7900a` and shares the same roaming requirements.

---

5. What Was Wrong (and Why It Had to Be Fixed)

The out-of-the-box Ubuntu 24.04 Wi-Fi configuration has several defaults that are fine for a laptop but harmful for a robot:

5.1 Wi-Fi Powersave Was On

Ubuntu installs `/etc/NetworkManager/conf.d/default-wifi-powersave-on.conf` with `wifi.powersave = 3`, which tells the driver to aggressively suspend the radio between packets.

Effect on robots: a sleeping radio introduces 50–200 ms of wake-up latency on every burst. ROS 2 DDS (UDP) transport times out nodes, causes dropped TF frames, and makes rviz2 visualization stutter. Under high-throughput video streaming, the radio could not sustain the required duty cycle and the connection dropped entirely.

Fix: `/etc/NetworkManager/conf.d/99-wifi-powersave-off.conf`

```
[connection]
wifi.powersave = 2
```

The `99-` prefix ensures this file is processed last and wins over the conflicting default. The conflicting `default-wifi-powersave-on.conf` was deleted.

Additionally, each connection profile sets `802-11-wireless.powersave = 2` so the setting survives even if the global conf.d files are reset.

5.2 PCIe ASPM Was Causing Link Resets (MT7922 Only)

The MT7922 chip on `sigyn7900a` uses a PCIe bus. Linux's PCIe Active State Power Management (ASPM) was allowed to clock-gate the PCIe link between packets, which caused the driver to lose sync with the firmware and reset — appearing as brief Wi-Fi disconnections.

Fix: `/etc/modprobe.d/mt7921e.conf`

```
options mt7921e disable_aspm=Y
```

This is applied at module load time. The Raspberry Pi's `brcmfmac` is SDIO-attached, not PCIe, so this fix is not needed on `sigynVision`.

5.3 The Radio Was Sticking to 2.4 GHz

Without explicit band configuration, the NetworkManager profile had no band preference. The device would associate with whichever BSSID responded first — often a 2.4 GHz radio (which responds more aggressively). The 2.4 GHz band tops out at ~130 Mbit/s link rate versus 720–960 Mbit/s on 5 GHz with the same AP.

Fix: `802-11-wireless.band = a` in both robot profiles. This forces the supplicant to only scan and associate on 5 GHz. The channel is left unpinned (`channel = 0`) so the robot can roam to any 5 GHz channel the mesh uses.

5.4 MAC Address Randomization Was Breaking Roaming

Ubuntu randomizes the MAC address used during Wi-Fi scanning by default. Some firmware builds do not distinguish scan MACs from association MACs cleanly; the mesh access points saw a different MAC arriving than the one from the previous association and treated it as a new client, resetting the session and dropping in-flight packets.

Fix: `mac-address-randomization = never` in both robot profiles and globally via `/etc/NetworkManager/conf.d/20-wifi-scan-rand-off.conf`:

```
[device]
wifi.scan-rand-mac-address = no
```

5.5 Internet Traffic Was Routing Via the TP-Link Adapter, Not Wi-Fi

The robot previously used a USB/PCIe TP-Link Ethernet adapter (`eno1`) as its primary network interface because the built-in Wi-Fi had poor throughput (caused by problems §5.1–5.4 above). After fixes were applied, `eno1` still held the default route (metric 100) while `wlp8s0` had no default route at all, meaning all traffic was still going through the old path.

Fix: set `ipv4.route-metric = 50` on the Wi-Fi profile and `ipv4.never-default = yes` on the `eno1` profile, then remove `eno1` from service. The Wi-Fi profile now owns the default route.

5.6 The Desktop Was on Wi-Fi

`amdc` was connected to the mesh over Wi-Fi. Video streaming from `sigyn7900a` to `amdc` went:

```
sigyn7900a → (Wi-Fi) → Nest AP → (5 GHz mesh backhaul) → Nest AP –
```

This means *two* over-the-air hops plus mesh backhaul contention. Measured throughput: **126 Mbps** with 101 TCP retransmits in 10 seconds.

After connecting `amdc` via Ethernet:

```
sigyn7900a → (Wi-Fi) → Nest AP → (wired) → amdc
```

Measured throughput: **340 Mbps** — 2.7× improvement, retransmits dropped proportionally. For ROS 2 DDS/UDP video topics, this translates directly to higher sustainable image rates and lower latency.

---

6. Global NetworkManager Tuning Files

These files exist on **all Wi-Fi machines** in the fleet (`sigyn7900a`, `sigynVision`):

`/etc/NetworkManager/conf.d/99-wifi-powersave-off.conf`

```
[connection]
wifi.powersave = 2
```

`/etc/NetworkManager/conf.d/20-wifi-scan-rand-off.conf`

```
[device]
wifi.scan-rand-mac-address = no
```

After creating or changing these files, reload NetworkManager:

```
sudo systemctl reload NetworkManager
```

---

7. The Roaming Agent

The mesh has 4 Google Nest repeaters spread through the house. The robot roams between all of them as it navigates. Without intervention, Linux's default roaming policy is conservative: it only roams when the signal is completely unusable (around -80 dBm), by which point MCS (modulation order) has already dropped from MCS 8/9 down to MCS 0/1, reducing throughput from ~800 Mbps to ~40 Mbps.

A systemd service (`wifi-roam-agent`) runs on both `sigyn7900a` and `sigynVision` to override this behavior.

How it works

Every 6 seconds the agent:

1. Reads the current BSSID, signal (dBm), and frequency from the driver.
2. Triggers a passive background scan.
3. Selects the strongest 5 GHz BSSID for the `livingroom` SSID from scan results.
4. If the current signal is on **2.4 GHz** and a 5 GHz AP is reachable at ≥ −72 dBm, it forces an immediate roam to the 5 GHz node.

5. If the current signal is ≤ **–55 dBm** and a better 5 GHz AP exists with at least **5 dB** improvement, it roams — subject to a 15-second minimum between roams to avoid thrashing.

The –55 dBm threshold is deliberately above the point where MCS degrades (~–65 dBm for MCS 7 at 80 MHz), so the robot pre-emptively hands off while throughput is still close to peak.

Service locations

```
/usr/local/sbin/wifi-roam-agent.sh          # the script
/etc/systemd/system/wifi-roam-agent.service # systemd unit
```

Environment variables (configurable in the service unit)

Variable	Default	Meaning
`IFACE`	`wlp8s0` / `wlan0`	Wi-Fi interface name
`TARGET_SSID`	`livingroom`	SSID to roam within
`LOW_SIGNAL_DBM`	`-55`	Roam trigger threshold (dBm)
`MIN_IMPROVEMENT_DB`	`5`	Minimum gain to justify a roam
`SCAN_INTERVAL`	`6`	Seconds between scan cycles
`MIN_ROAM_INTERVAL`	`15`	Minimum seconds between roams

Deploying to a new robot machine

```
# On your development machine (assuming passwordless sudo SSH):
scp /usr/local/sbin/wifi-roam-agent.sh $NEW_ROBOT:/tmp/
ssh -t $NEW_ROBOT "sudo cp /tmp/wifi-roam-agent.sh /usr/local/sbin
  sudo chmod +x /usr/local/sbin/wifi-roam-agent.sh"

# Create /etc/systemd/system/wifi-roam-agent.service on the new ma
# Change IFACE to match the interface name (wlp8s0 or wlan0, etc.)
# Then:
ssh -t $NEW_ROBOT "sudo systemctl daemon-reload && \
  sudo systemctl enable --now wifi-roam-agent"
```

Check logs:

```
journalctl -u wifi-roam-agent -f
```

---

8. Setting Up a New Robot Machine

When adding a computer to the fleet:

1. **Set a static IP** in the NetworkManager profile:

   ```
   nmcli con modify <profile-name> \
     ipv4.method manual \
     ipv4.addresses <NEW_IP>/24 \
     ipv4.gateway 192.168.12.1 \
     ipv4.dns "8.8.8.8 8.8.4.4" \
     ipv4.route-metric 50 \
     ipv4.never-default no
   ```

2. **Force 5 GHz, disable powersave and MAC randomization:**

   ```
   nmcli con modify <profile-name> \
     802-11-wireless.band a \
     802-11-wireless.powersave 2 \
     802-11-wireless.mac-address-randomization never
   ```

3. **Add the global NM conf.d files** (see §6).

4. **Delete the conflicting default powersave file** if present:

   ```
   sudo rm -f /etc/NetworkManager/conf.d/default-wifi-powersave-
   sudo systemctl reload NetworkManager
   ```

5. **Add the PCIe ASPM fix** if the machine uses the MT7922 or similar MediaTek PCIe chip:

   ```
   echo "options mt7921e disable_aspm=Y" | sudo tee /etc/modprob
   sudo update-initramfs -u
   ```

6. **Update `/etc/hosts`** on every machine in the fleet with the new entry.

7. **Deploy the roaming agent** (see §7).

8. **Apply and reboot:**

   ```
   nmcli con up <profile-name>
   ```

---

9. Benchmarking

Verify LAN throughput after setting up a machine. Run on `amdc` (wired):

```
iperf3 -s
```

Run on the robot:

```
iperf3 -c amdc -t 10 -P 4
```

Expected results at good signal (–50 dBm, MCS 8, 2×2 MIMO, 80 MHz):

- **Before fixes (typical):** 10–30 Mbps, many retransmits
- **After fixes:** 300–400 Mbps, few retransmits

If you see unexpectedly low throughput, check:

```
iw dev <iface> link          # look at tx/rx bitrate and MCS index
journalctl -u wifi-roam-agent -n 20   # recent roam events
nmcli -f ACTIVE,BSSID,SSID,FREQ,SIGNAL,RATE dev wifi   # current A
```

---

10. Quick Reference

```
# Check current Wi-Fi link quality
iw dev wlp8s0 link

# See all visible APs and their signal/band
nmcli dev wifi list ifname wlp8s0

# Show active connection profile
nmcli con show --active

# Force reconnect (picks up any profile changes)
nmcli con up livingroom

# Roam agent status and recent events
systemctl status wifi-roam-agent
journalctl -u wifi-roam-agent -n 30

# Verify static IP is applied
ip -4 addr show wlp8s0

# Verify default route goes via Wi-Fi
ip -4 route get 1.1.1.1
```